

Mini Project 1: Investigating OTA Firmware Integrity Using MQTT and Merkle Trees

Objective

Over-the-Air (OTA) updates allow software and firmware to be distributed to remote devices without physical access. However, an update may be corrupted during transmission or deliberately modified by an attacker.

In this mini-project, you will design, implement, and evaluate a simplified OTA firmware-update system. A workstation will distribute a firmware file, a Raspberry Pi 4 will act as the target device, and MQTT will provide the communication between them. You will use a Merkle tree to verify the integrity of the received firmware.

A text file will represent the firmware. You must not install or modify actual Raspberry Pi firmware. All testing must occur on isolated lab resources you fully control.

Main question: How can Merkle-tree verification detect tampering with firmware chunks during an MQTT-based OTA update, and what are the limitations of this verification?

You must answer this question using your implementation, experimental results, and security analysis.

1. Configure the MQTT environment

Connect the workstation and Raspberry Pi 4 to the same local network. Install and configure an MQTT broker (e.g., Mosquitto, HiveMQ) on the local network.

Before implementing the OTA update, demonstrate that:

1. The workstation can publish an MQTT message.
2. The Raspberry Pi can subscribe to the corresponding topic.
3. The Raspberry Pi receives the published message.

In the report, identify the publisher, subscriber, broker, and topics in your system. Select an MQTT Quality of Service level for the OTA messages and justify the selection.

2. Design the Server-Side Chunking and Manifest Generator

Create a Python server script that reads a firmware text file (firmware.txt) and prepares it for distribution:

- Divide the complete file into exactly four ordered, non-empty byte chunks: C_0, C_1, C_2, C_3
- Define how the file is divided when its size is not divisible by four.
- Calculate the SHA-256 hash of each chunk to obtain the four leaf nodes: H_0, H_1, H_2, H_3
- Construct a binary Merkle tree by combining the leaf hashes in ordered pairs and hashing each pair to obtain the parent nodes. Continue this process until a single Merkle root is produced.
- Generate a manifest containing the firmware version, original firmware size, total number of chunks, Merkle root, and the index and filename of each chunk.

3. Design and Implement Firmware Distribution over MQTT

Design an MQTT publish–subscribe flow in which the workstation publishes the manifest and firmware chunks, while the Raspberry Pi subscribes to the relevant topics. Implement the publishing functionality in `ota_server.py`.

You should decide and explain:

- The MQTT topic structure
- The payload format used for the manifest and chunks
- How binary chunk data is represented
- How each chunk is associated with its firmware version and index
- The MQTT Quality of Service level
- The order in which the publisher and subscriber must be started
- How duplicated, missing, delayed, or reordered messages are handled

4. Implement the Raspberry Pi OTA Client

Implement `ota_client.py` on the Raspberry Pi.

The client must:

- Connect to the MQTT broker.
- Subscribe to the manifest and firmware-chunk topics.
- Receive and interpret the manifest.
- Receive the four firmware chunks.
- Store each chunk according to its index rather than its arrival order.
- Handle duplicate messages without counting the same chunk more than once.
- Apply a timeout if one or more chunks are not received.
- Confirm that the received chunk indices are exactly 0, 1, 2, and 3.
- Calculate a Merkle root from the received chunks.
- Compare the calculated root with the reference root contained in the manifest.

If all four chunks are present, the roots match, and the reconstructed size matches the size stated in the manifest, the client must reconstruct the chunks in index order and create `firmware_reconstructed.txt`.

If any check fails, the client must reject the update, clear the received chunks, record the reason for failure, and ensure that `firmware_reconstructed.txt` is not created.

6. Investigate Attacks Against OTA Updates

Investigate two attacks against OTA firmware updates. At least one attack must consider manipulation or replacement of the update manifest.

For each attack:

1. Describe the attacker's action.
2. Identify which part of the update process is affected.
3. Determine whether your implementation would detect the attack.
4. Justify your conclusion.
5. Identify any additional protection that would be required.

Deliverables - 1 zip file, 2 folders (code and report)

Source code:

- All your code must be in 3 scripts + 1 text file: the server (ota_server.py), the Raspberry Pi client (ota_client.py) and the orchestrator (run.py), also include your firmware file (firmware.txt).
- Add a README file if your script requires extra arguments or special instructions to run.

Report:

- The first page should be a cover page that includes your names, course title, date, master's program title, and university name.
- Answer all questions in order, using the correct numbering format provided.
- Include key code snippets and configuration files (only the essential ones).

Please use a justified text format and ensure that your screenshots are readable in the final PDF.